Machine Learning

Author: Isaac FEI

Contents

I Elementary Concepts **1**

Chapter 1 Singular Value Decomposition (SVD) **2**

1.1 A Geometric Interpretation 2

1.2 Image Compression 2

1.3 Optimal Hard Threshold for Singular Values 5

1.4 Face Recognition 5

1.5 Edge Detection: An Example of Complex-Valued Datasets 5

Chapter 2 Dimensionality Reduction **6**

2.1 Curse of Dimensionality 6

2.2 Principal Component Analysis (PCA) 6

Part I

Elementary Concepts

Chapter 1 Singular Value Decomposition (SVD)

1.1 A Geometric Interpretation

Theorem 1.1.1

Let A be an $m \times n$ real matrix with rank r. Then A can be decomposed by

$$A = U\Sigma V^\top$$

where U and V are orthogonal matrices with shape $m \times m$ and $n \times n$, respectively. Σ is an $m \times n$ matrix in the form of

$$\left[\begin{array}{ccc|c} \sigma_1 & & & \\ & \ddots & & 0 \\ & & \sigma_r & \\ \hline & 0 & & 0 \end{array}\right]$$

where $\sigma_1, \ldots, \sigma_r$ are nonnegative numbers satisfying

$$\sigma_1 \geq \cdots \geq \sigma_r > 0$$

♡

Proof ■

1.2 Image Compression

Given a color image with height m (pixels) and width n (pixels), we first split the RGB channels. Then, each channel is regarded as an $m \times n$ matrix A. Applying the SVD to A, we obtain

$$A = U\Sigma V^\top$$

In practice, A is very likely to have full rank, which means it has as many singular values as possible (at most $\min\{m, n\}$ singular values). But not all singular values along with its associated left and right singular vectors are of the same importance to the image, which we will explain in the following way.

For a pixel a_{ij} located at the i-th row and j-th column in the image A, its value equals to

$$a_{ij} = \sum_{k=1}^{n} \sigma_k u_{ik} v_{jk}$$

If the singular value σ_k becomes very small starting from $k \geq r+1$, then the sum $\sum_{k=r+1}^{n} \sigma_k u_{ik} v_{jk}$ has little contribution to the pixel value a_{ij}. Hence, a_{ij} can be approximated by

$$a_{ij} \approx \sum_{k=1}^{r} \sigma_k u_{ik} v_{jk}$$

In this way, the original image A can be compressed by only keeping the first r singular values

while dropping the rest. Mathematically, the compressed image $\hat{A}$ is given by

$$\hat{A} = \hat{U}\hat{\Sigma}\hat{V}^\top = \begin{bmatrix} \mathbf{u}_1 & \cdots & \mathbf{u}_r \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_r^\top \end{bmatrix}$$

Recall A represents a single channel. The final compressed color image can be then generated by merging all three compressed images from each channel.

We now demonstrate the algorithm to compress the image shown in Figure 1.1 in Python.

Figure 1.1: Mountain

The Python code is listed below. The parameter `r` in function `compress_image` is the number of singular values we want to keep after compression.

```python
# open image file
img: Image.Image = Image.open("./data/mountain.jpg")

def compress_channel(A: np.ndarray, r: int) -> np.ndarray:
    # SVD
    U, s, Vh = svd(A)

    # keep first r singular values
    U = U[:, :r]
    s = s[:r]
    Vh = Vh[:r, :]
    A_hat = (U * s) @ Vh

    # clip pixel values
    A_hat = np.clip(A_hat, a_min = 0, a_max = 255)
    A_hat = A_hat.astype(np.uint8)

    return A_hat

def compress_image(img: Image.Image, r: int) -> Image.Image:
    # split RGB channels
    channels: list[Image.Image] = img.split()

    compressed_channels = []
    for channel in channels:
```

```
26        # convert image to NumPy array
27        A = np.array(channel)
28
29        # compress single channel
30        A_hat = compress_channel(A, r)
31
32        # convert to Image class instance
33        channel = Image.fromarray(A_hat)
34        compressed_channels.append(channel)
35
36    # merge channels
37    compressed_img = Image.merge("RGB", compressed_channels)
38
39    return compressed_img
```

Note *It is important to clip the pixel values as in line 15. If this line is omitted, then the resulting compressed image is not as expected.*

Figure 1.2 shows the result of the compressed images with `r = 10` and `r = 50`. The size of the original image is 380 KB. After compression, the size reduces to 97 KB and 154 KB, respectively.

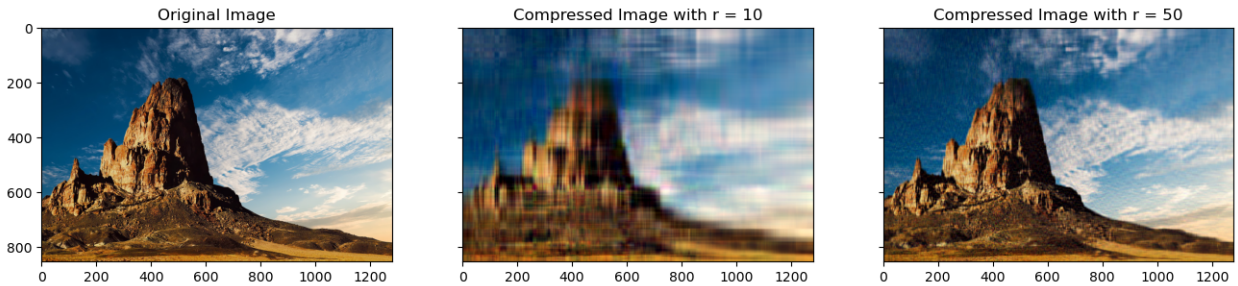

Figure 1.2: Original Image and Its Compressed Images

When `r` is 10, the compressed image is rather vague. But when `r` rises to 50, the decline in image quality is seamless.

Proposition 1.2.1

Suppose $A \in \mathbb{F}^{m\times n}$, $B \in \mathbb{F}^{n\times k}$ and $C \in \mathbb{F}^{l\times n}$ where $\mathbb{F}$ is either $\mathbb{R}$ or $\mathbb{C}$. Then we have the following properties:

1. *If* $\operatorname{rank}(B) = n$, *then* $\operatorname{rank}(AB) = \operatorname{rank}(A)$.
2. *If* $\operatorname{rank}(C) = m$, *then* $\operatorname{rank}(CA) = \operatorname{rank}(A)$.

Remark One may suggest replacing the condition $\operatorname{rank}(B) = n$ with that B has full rank in statement 1. But in general, that is not true. For example, suppose A is a 2×2 invertible matrix and B is a non-zero column vector. Then it is clear that A has rank 2 and B has full rank (rank 1). But AB is a column vector, which has rank 1 instead of 2. Similarly, we cannot replace the condition $\operatorname{rank}(C) = m$ with that C has full rank in statement 2.

1.3 Optimal Hard Threshold for Singular Values

1.4 Face Recognition

1.5 Edge Detection: An Example of Complex-Valued Datasets

Chapter 2 Dimensionality Reduction

2.1 Curse of Dimensionality

2.2 Principal Component Analysis (PCA)

Definition 2.2.1

Let $\mathbf{X} = [X_1 \cdots X_n]^\top$ be an n-dimentional vector of random variables. Suppose $\mathbf{v}_1, \ldots, \mathbf{v}_n$ are n unit vectors. Define random variable

$$T_j = \mathbf{v}_j^\top \mathbf{X} \quad 1 \le j \le n$$

*Then we say $\mathbf{v}_j$ is the j-th **principal component** of $\mathbf{X}$ if $\mathbf{v}_j$ is the solution to the following optimization problem:*

$$\max_{\mathbf{v}_j} \quad \mathrm{var}(T_j) \tag{2.1}$$

$$\textit{s.t.} \quad \begin{cases} \mathbf{v}_j^\top \mathbf{v}_j = 1 \\ \mathrm{cov}(T_i, T_j) = 0 \quad \forall 1 \le i \le j-1 \end{cases}$$

*In this case, T_j is called the **principal component score** for $\mathbf{v}_j$.* ♣

Remark Note that the principal component scores $T_1, \ldots, T_n$ are mutually uncorrelated, i.e.,

$$i \neq j \implies \mathrm{cov}(T_i, T_j) = 0$$

Now, we have formulated the rigorous definition of the principal components, which captures the idea of preserving the maximum variance of the data. But what are the principal components exactly, and how can we find them? As stated in the above definition, the principal component $\mathbf{v}_j$ can be found by solving the optimization problem (2.1).

The following theorem provides the solution to (2.1). To our surprise, the j-th principal component is the j-th eigenvector of the covariance matrix $\mathrm{cov}(\mathbf{X}, \mathbf{X})$ if the eigenvalues are sorted in the decreasing order.

Theorem 2.2.1

Suppose that $\mathbf{X} = [X_1 \cdots X_n]^\top$ is an n-dimentional vector of random variables. Let $\Sigma = \mathrm{cov}(\mathbf{X}, \mathbf{X})$ denote the covariance matrix. Since Σ is positive semi-definite, it has n nonnegative eigenvalues, say $\lambda_1, \ldots, \lambda_n$. We may assume that these eigenvalues are sorted in the descending order, i.e.,

$$\lambda_1 \ge \cdots \ge \lambda_n \ge 0$$

Let $\mathbf{v}_1, \ldots, \mathbf{v}_n$ be the corresponding orthonormal eigenvectors. Then $\mathbf{v}_j$ is precisely the j-th principal component of $\mathbf{X}$. ♡

The general idea of this proof is to solve (2.1) with the method of Lagrange multiplier.

Proof (Covariance as Matrix multiplications) We first write $\mathrm{cov}(T_i, T_j)$ in the form of matrix multiplications. We have

$$\begin{aligned}
\mathrm{cov}(T_i, T_j) &= \mathrm{cov}\left(\mathbf{v}_i^\top \mathbf{X}, \mathbf{v}_j^\top \mathbf{X}\right) \\
&= \mathrm{cov}\left(\sum_{k=1}^{n} v_{ik} X_k, \sum_{l=1}^{n} v_{jl} X_l\right) \\
&= \sum_{k=1}^{n}\sum_{l=1}^{n} v_{ik} v_{jl}\, \mathrm{cov}(X_k, X_l) \\
&= \mathbf{v}_i^\top \Sigma \mathbf{v}_j
\end{aligned}$$

In summary,

$$\mathrm{cov}(T_i, T_j) = \mathbf{v}_i^\top \Sigma \mathbf{v}_j$$

Specially, when $i = j$ we have

$$\mathrm{var}(T_j) = \mathbf{v}_j^\top \Sigma \mathbf{v}_j$$

(Induction Hypothesis) We shall prove this theorem by induction. The induction hypothesis is stated as follows.

$$P(m) : \mathbf{v}_1, \ldots, \mathbf{v}_m \text{ are the first } m \text{ principal components.} \tag{2.2}$$

(Base Case: The First Principal Component) Consider the optimization problem (2.1) for $j = 1$:

$$\begin{aligned}
\max_{\mathbf{v}} \quad & \mathrm{var}(T) = \mathrm{var}(\mathbf{v}^\top \mathbf{X}) = \mathbf{v}^\top \Sigma \mathbf{v} \\
\text{s.t.} \quad & \mathbf{v}^\top \mathbf{v} = 1
\end{aligned} \tag{2.3}$$

Note that in this case, there is only one constraint $\mathbf{v}^\top \mathbf{v} = 1$. The Lagrangian function is given by

$$\mathcal{L}(\mathbf{v}; \lambda) = \mathbf{v}_1^\top \Sigma \mathbf{v} - \lambda \mathbf{v}^\top \mathbf{v}$$

Taking the partial derivative with respect to $\mathbf{v}$, we obtain

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}} = 2\Sigma \mathbf{v} - 2\lambda \mathbf{v}$$

Letting the partial derivative equal to zero, it follows that

$$(\Sigma - \lambda I)\mathbf{v} = \mathbf{0}$$

This implies λ is an eigenvalue of Σ and $\mathbf{v}$ is its corresponding eigenvector. Then, the variance we want to maximize in (2.3) reduces to

$$\mathrm{var}(T) = \mathbf{v}^\top \Sigma \mathbf{v} = \lambda \mathbf{v}^\top \mathbf{v} = \lambda$$

Therefore, the maximum of $\mathrm{var}(T) = \lambda$ is λ_1. And then the optimal solution $\mathbf{v}_1$ is the eigenvector corresponding to λ_1.

(Inductive Step) Assume the induction hypothesis (2.2) holds for k, i.e., $P(k)$ holds. Now, consider the optimization problem:

$$\begin{aligned}
\max_{\mathbf{v}} \quad & \mathrm{var}(T) = \mathrm{var}(\mathbf{v}^\top \mathbf{X}) = \mathbf{v}^\top \Sigma \mathbf{v} \\
\text{s.t.} \quad & \begin{cases} \mathbf{v}^\top \mathbf{v} = 1 \\ \mathrm{cov}(T_i, T) = 0 \quad \forall 1 \le i \le k \end{cases}
\end{aligned} \tag{2.4}$$

Recall we have shown $\mathrm{cov}(T_i, T) = \mathbf{v}_i^\top \Sigma \mathbf{v} = \mathbf{v}^\top \Sigma \mathbf{v}_i$. And since $\mathbf{v}_i$ is an eigenvector corresponding to λ_i, it follows that

$$\mathrm{cov}(T_i, T) = \mathbf{v}^\top \Sigma \mathbf{v}_i = \lambda_i \mathbf{v}^\top \mathbf{v}_i = \lambda_i \mathbf{v}_i^\top \mathbf{v}$$

Therefore, the problem (2.4) is reduced to

$$\max_{\mathbf{v}} \quad \mathrm{var}(T) = \mathrm{var}(\mathbf{v}^\top \mathbf{X}) = \mathbf{v}^\top \Sigma \mathbf{v} \tag{2.5}$$

$$\text{s.t.} \quad \begin{cases} \mathbf{v}^\top \mathbf{v} = 1 \\ \lambda_i \mathbf{v}_i^\top \mathbf{v} = 0 \quad \forall 1 \le i \le k \end{cases}$$

The Lagrangian function is then given by

$$\mathcal{L}(\mathbf{v}; \lambda, \phi_1, \ldots, \phi_k) = \mathbf{v}^\top \Sigma \mathbf{v} - \lambda(\mathbf{v}^\top \mathbf{v} - 1) - \phi_1 \lambda_1 \mathbf{v}_1^\top \mathbf{v} - \cdots - \phi_k \lambda_k \mathbf{v}_k^\top \mathbf{v}$$

The partial derivative of $\mathbf{v}$ is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}} = 2\Sigma \mathbf{v} - 2\lambda \mathbf{v} - \phi_1 \lambda_1 \mathbf{v}_1 - \cdots - \phi_k \lambda_k \mathbf{v}_k$$

$\frac{\partial \mathcal{L}}{\partial \mathbf{v}} = \mathbf{0}$ yields that

$$(\Sigma - \lambda I)\mathbf{v} = \frac{1}{2}(\phi_1 \lambda_1 \mathbf{v}_1 + \cdots + \phi_k \lambda_k \mathbf{v}_k) \tag{2.6}$$

For each vector $\mathbf{v}_j$ $(1 \le j \le k)$, we have

$$\mathbf{v}_i^\top \mathbf{v} = 0$$

due to the constraint of (2.5). Moreover,

$$\mathbf{v}_i^\top \mathbf{v}_j = \begin{cases} 1 & i = j \\ 0 & i \ne j \end{cases}$$

due to the hypothesis. Then Taking the inner product with $\mathbf{v}_i$ on both sides of (2.6), we obtain

$$0 = (\Sigma - \lambda I)\mathbf{v}_i^\top \mathbf{v} = \frac{1}{2}(\phi_1 \lambda_1 \mathbf{v}_i^\top \mathbf{v}_1 + \cdots + \phi_k \lambda_k \mathbf{v}_i^\top \mathbf{v}_k) = \frac{1}{2}\phi_i \lambda_i \quad \forall 1 \le i \le k \tag{2.7}$$

Equation (2.7) yields that

$$\phi_i \lambda_i = 0 \quad \forall 1 \le i \le k$$

Then (2.6) reduces to

$$(\Sigma - \lambda I)\mathbf{v} = \mathbf{0}$$

This implies λ is an eigenvalue of Σ and $\mathbf{v}$ is its corresponding eigenvector. Recall problem (2.5). We want to maximize

$$\mathrm{var}(T) = \mathbf{v}^\top \Sigma \mathbf{v} = \lambda \mathbf{v}^\top \mathbf{v} = \lambda$$

Hence, we want λ to be as large as possible. And the optimal choice is $\lambda = \lambda_{k+1}$ because λ cannot be λ_i $(1 \le i \le k)$ if $\lambda_i > \lambda_{k+1}$. Otherwise, $\lambda = \lambda_i$ would imply that $\mathbf{v}$ being an eigenvector of λ_i, which would lead to a contradiction since all the independent eigenvectors are already included among $\mathbf{v}_1, \ldots, \mathbf{v}_k$. In conclusion, the solution to problem (2.5) is $\mathbf{v} = \mathbf{v}_{k+1}$.

This completes the proof. ■